

Principles of Programming Languages

Lecture 4: Semantics

Andrei Arusoaie¹

¹Department of Computer Science

October 25, 2019

Outline

Semantics: introduction

A simple imperative language

Semantics: motivation

C

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
4
```

Java

```
-$ cat File.java
public class File {
    ... void main(...) {
        int x = 0;
        println((x=1) + (x=2));
    }
}
-$ javac File.java
-$ java File
3
```

Semantics: motivation

C

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
```

4

Java

```
-$ cat File.java
public class File {
    ... void main(...) {
        int x = 0;
        println((x=1) + (x=2));
    }
}
-$ javac File.java
-$ java File
```

3

Semantics: motivation

C

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
4
```

Java

```
-$ cat File.java
public class File {
    ... void main(...) {
        int x = 0;
        println((x=1) + (x=2));
    }
}
-$ javac File.java
-$ java File
3
```

Semantics: motivation

GCC: 5.4.0-6 ubuntu

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
4
```

GCC: LLVM version 9.0.0 - clang

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
3
```

Semantics: motivation

GCC: 5.4.0-6 ubuntu

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
```

4

GCC: LLVM version 9.0.0 - clang

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
```

3

Semantics: motivation

GCC: 5.4.0-6 ubuntu

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
4
```

GCC: LLVM version 9.0.0 - clang

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
3
```


Semantics: motivation

GCC: 5.4.0-6 ubuntu

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
4
```

GCC: LLVM version 9.0.0 - clang

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
3
```

Demo

▶ `out-of-lifetime.c`

Semantics

- ▶ Semantics is concerned with the **meaning** of language constructs
- ▶ Semantics must be **unambiguous**
- ▶ Semantics must be **flexible**

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic styles

Some (formal) semantics styles:

- ▶ operational
- ▶ denotational
- ▶ axiomatic

We will focus more on **operational** semantics styles: Small-step SOS, Big-Step SOS

Semantic styles

Some (formal) semantics styles:

- ▶ operational
- ▶ denotational
- ▶ axiomatic

We will focus more on **operational** semantics styles: Small-step SOS, Big-Step SOS

A simple imperative language

Program:

```
n ::= 10;;  
i ::= 1;;  
sum ::= 0;;  
while (i <= n) do  
    sum ::= sum + ' i;;  
    i ::= i + ' 1  
end
```

Features:

- ▶ variables
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

A simple imperative language

Program:

```
n ::= 10;;  
i ::= 1;;  
sum ::= 0;;  
while (i <= n) do  
    sum ::= sum + i;;  
    i ::= i + 1  
end
```

Features:

- ▶ variables
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

A simple imperative language

Program:

```
n ::= 10;;  
i ::= 1;;  
sum ::= 0;;  
while (i <= n) do  
    sum ::= sum + ' i;;  
    i ::= i + ' 1  
end
```

Features:

- ▶ **variables**
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

A simple imperative language

Program:

```
n ::= 10;;  
i ::= 1;;  
sum ::= 0;;  
while (i <= n) do  
    sum ::= sum + ' i;;  
    i ::= i + ' 1  
end
```

Features:

- ▶ variables
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

A simple imperative language

Program:

```
n ::= 10;;  
i ::= 1;;  
sum ::= 0;;  
while (i <= n) do  
    sum ::= sum + ' i;;  
    i ::= i + ' 1  
end
```

Features:

- ▶ variables
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

A simple imperative language

Program:

```
n ::= 10;;  
i ::= 1;;  
sum ::= 0;;  
while (i <= n) do  
    sum ::= sum + i;;  
    i ::= i + 1  
end
```

Features:

- ▶ variables
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

A simple imperative language

Program:

```
n ::= 10;;  
i ::= 1;;  
sum ::= 0;;  
while (i <= n) do  
    sum ::= sum + ' i;;  
    i ::= i + ' 1  
end
```

Features:

- ▶ variables
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

A simple imperative language

Program:

```
n ::= 10;;  
i ::= 1;;  
sum ::= 0;;  
while (i <= n) do  
    sum ::= sum + i;;  
    i ::= i + 1  
end
```

Features:

- ▶ variables
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

A simple imperative language

Program:

```
n ::= 10;;  
i ::= 1;;  
sum ::= 0;;  
while (i <= n) do  
    sum ::= sum + i;;  
    i ::= i + 1  
end
```

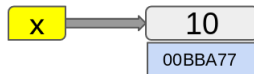
Features:

- ▶ variables
- ▶ arithmetic expressions
- ▶ boolean expressions
- ▶ assignment statements
- ▶ loop statements
- ▶ decisional statements

How can we define the semantics of this language?

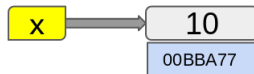
Variables

- ▶ variable = *storage location at address* + *symbolic name*
- ▶ variables can be *accessed* or *modified*
- ▶ depending on the language, they may only be able to store a specified datatype (e.g., integer, string, etc.)
- ▶ *Scope*: global, local
- ▶ *Lifetime (extent)*
- ▶ $var_name \mapsto value$
- ▶ Environment + memory



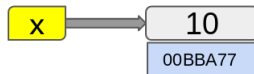
Variables

- ▶ variable = *storage location* at address + *symbolic name*
- ▶ variables can be *accessed* or *modified*
- ▶ depending on the language, they may only be able to store a specified datatype (e.g., integer, string, etc.)
- ▶ *Scope*: global, local
- ▶ *Lifetime (extent)*
- ▶ $var_name \mapsto value$
- ▶ Environment + memory



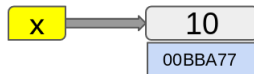
Variables

- ▶ variable = *storage location at address* + *symbolic name*
- ▶ variables can be *accessed* or *modified*
- ▶ depending on the language, they may only be able to store a specified datatype (e.g., integer, string, etc.)
- ▶ *Scope*: global, local
- ▶ *Lifetime (extent)*
- ▶ *var_name* $\mapsto$ *value*
- ▶ Environment + memory



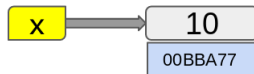
Variables

- ▶ variable = *storage location* at *address* + *symbolic name*
- ▶ variables can be *accessed* or *modified*
- ▶ depending on the language, they may only be able to store a specified datatype (e.g., integer, string, etc.)
- ▶ *Scope*: global, local
- ▶ *Lifetime (extent)*
- ▶ *var_name* $\mapsto$ *value*
- ▶ *Environment + memory*



Variables

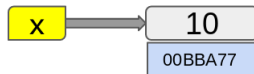
- ▶ variable = *storage location* at address + *symbolic name*
- ▶ variables can be *accessed* or *modified*
- ▶ depending on the language, they may only be able to store a specified datatype (e.g., integer, string, etc.)
- ▶ *Scope*: global, local
- ▶ *Lifetime* (*extent*)
- ▶ *var_name* $\mapsto$ *value*
- ▶ Environment + memory



Variables

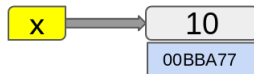
- ▶ variable = *storage location* at address + *symbolic name*
- ▶ variables can be *accessed* or *modified*
- ▶ depending on the language, they may only be able to store a specified datatype (e.g., integer, string, etc.)
- ▶ *Scope*: global, local
- ▶ *Lifetime* (*extent*)
- ▶ $var_name \mapsto value$

▶ Environment + memory



Variables

- ▶ variable = *storage location* at address + *symbolic name*
- ▶ variables can be *accessed* or *modified*
- ▶ depending on the language, they may only be able to store a specified datatype (e.g., integer, string, etc.)
- ▶ *Scope*: global, local
- ▶ *Lifetime* (*extent*)
- ▶ $var_name \mapsto value$
- ▶ Environment + memory



Variables in Coq

► Declaration:

```
Inductive Var := n | i | sum.
```

```
Definition var_eq (v1 v2 : Var) :=
```

```
match v1, v2 with
```

```
  | n, n => true
```

```
  | i, i => true
```

```
  | sum, sum => true
```

```
  | _, _ => false
```

```
end.
```

Environment

- Environment: variables mapped to values

```
Definition Env := Var -> nat.
```

```
Definition env (var : Var) := if (var_eq var n)
                                then 10
                                else 0.
```

```
Compute (env n).
```

```
= 10
```

```
: nat
```

```
Compute (env i).
```

```
= 0
```

```
: nat
```

```
Check env.
```

```
env:
```

```
      : Var -> nat
```

- The environment is a function!

Environment update

- Update = a new function s.t. the value for x is updated to v :

```
Definition update (x : Var) (v : nat) (env : Env) : Env :=  
  fun y => if (var_eq x y)  
    then v  
    else env y.
```

Arithmetic expressions

► Syntax:

```
BNF:
  AExp ::=  N
        |  AExp '+' AExp
        |  AExp '**' AExp
```

```
Inductive AExp :=
| anum : nat -> AExp
| aplus : AExp -> AExp -> AExp
| amul : AExp -> AExp -> AExp.
```

► Semantics:

```
Fixpoint aeval (a : AExp) : nat :=
match a with
| anum v => v
| aplus a1 a2 => (aeval a1) + (aeval a2)
| amul a1 a2 => (aeval a1) * (aeval a2)
end.
```

► What about expressions in a PL: $i ::= \underline{i + 1}; ; ?$

Arithmetic expressions

► Syntax:

```
BNF:
AExp ::=  N
        |  AExp '+' AExp
        |  AExp '**' AExp
```

```
Inductive AExp :=
| anum : nat -> AExp
| aplus : AExp -> AExp -> AExp
| amul : AExp -> AExp -> AExp.
```

► Semantics:

```
Fixpoint aeval (a : AExp) : nat :=
match a with
| anum v => v
| aplus a1 a2 => (aeval a1) + (aeval a2)
| amul a1 a2 => (aeval a1) * (aeval a2)
end.
```

► What about expressions in a PL: $i ::= \underline{i + 1}; ; ?$

Arithmetic expressions

► Syntax:

BNF:		Inductive AExp :=
AExp ::=	$\mathbb{N}$	anum : nat -> AExp
	AExp '+' AExp	aplus : AExp -> AExp -> AExp
	AExp '*' AExp	amul : AExp -> AExp -> AExp.

► Semantics:

```
Fixpoint aeval (a : AExp) : nat :=
  match a with
  | anum v => v
  | aplus a1 a2 => (aeval a1) + (aeval a2)
  | amul a1 a2 => (aeval a1) * (aeval a2)
  end.
```

► What about expressions in a PL: $i ::= \underline{i + 1}; ; ?$

Arithmetic expressions

► Syntax:

BNF:		Inductive AExp :=
AExp ::=	$\mathbb{N}$	anum : nat -> AExp
	AExp '+' AExp	aplus : AExp -> AExp -> AExp
	AExp '*' AExp	amul : AExp -> AExp -> AExp.

► Semantics:

```
Fixpoint aeval (a : AExp) : nat :=
  match a with
  | anum v => v
  | aplus a1 a2 => (aeval a1) + (aeval a2)
  | amul a1 a2 => (aeval a1) * (aeval a2)
  end.
```

► What about expressions in a PL: $i ::= \underline{i + 1}; ; ?$

Arithmetic expressions with variables

► Syntax:

BNF:		Inductive AExp :=
AExp ::=	Var	var : Var -> AExp
	$\mathbb{N}$	anum : nat -> AExp
	AExp '+' AExp	aplus : AExp -> AExp -> AExp
	AExp '*' AExp	amul : AExp -> AExp -> AExp.

► Semantics:

```
Fixpoint aeval (a : AExp) : nat :=
  match a with
  | var v => ???
  | anum v => v
  | aplus a1 a2 => (aeval a1) + (aeval a2)
  | amul a1 a2 => (aeval a1) * (aeval a2)
  end.
```

► We need an environment to evaluate expressions with variables!

Arithmetic expressions with variables

► Syntax:

BNF:		Inductive AExp :=
AExp ::=	Var	var : Var -> AExp
	$\mathbb{N}$	anum : nat -> AExp
	AExp '+' AExp	aplus : AExp -> AExp -> AExp
	AExp '*' AExp	amul : AExp -> AExp -> AExp.

► Semantics:

```
Fixpoint aeval (a : AExp) : nat :=  
  match a with  
  | var v => ???  
  | anum v => v  
  | aplus a1 a2 => (aeval a1) + (aeval a2)  
  | amul a1 a2 => (aeval a1) * (aeval a2)  
  end.
```

► We need an environment to evaluate expressions with variables!

Arithmetic expressions with variables

► Syntax:

BNF:		Inductive AExp :=
AExp ::=	Var	var : Var -> AExp
	$\mathbb{N}$	anum : nat -> AExp
	AExp '+' AExp	aplus : AExp -> AExp -> AExp
	AExp '*' AExp	amul : AExp -> AExp -> AExp.

► Semantics:

```
Fixpoint aeval (a : AExp) : nat :=
match a with
| var v => ???
| anum v => v
| aplus a1 a2 => (aeval a1) + (aeval a2)
| amul a1 a2 => (aeval a1) * (aeval a2)
end.
```

► We need an environment to evaluate expressions with variables!

Arithmetic expressions with variables

► Syntax:

BNF:		Inductive AExp :=
AExp ::=	Var	var : Var -> AExp
	$\mathbb{N}$	anum : nat -> AExp
	AExp '+' AExp	aplus : AExp -> AExp -> AExp
	AExp '*' AExp	amul : AExp -> AExp -> AExp.

► Semantics:

```
Fixpoint aeval (a : AExp) : nat :=  
match a with  
| var v => ???  
| anum v => v  
| aplus a1 a2 => (aeval a1) + (aeval a2)  
| amul a1 a2 => (aeval a1) * (aeval a2)  
end.
```

► We need an environment to evaluate expressions with variables!

Evaluate expression with variables

► Semantics:

```
Fixpoint aeval (a : AExp) (env : Env) : nat :=
  match a with
  | var v => env v
  | anum v => v
  | aplus a1 a2 => (aeval a1 env) + (aeval a2 env)
  | amul a1 a2 => (aeval a1 env) * (aeval a2 env)
  end.
```

► Notations and testing:

```
Coercion var : Var -> AExp.
Coercion anum : nat -> AExp.
Notation "A +' B" := (aplus A B) (at level 49).
Notation "A *' B" := (amul A B) (at level 48).
Compute aeval (2 +' 3 *' 4) env.
= 14
: nat
Compute env n.
= 10
: nat
Compute aeval (2 +' 3 *' n) env.
= 32
: nat
```


Boolean expressions

► Syntax:

```
Inductive BExp :=  
  | btrue  : BExp  
  | bfalse : BExp  
  | blessthan : AExp -> AExp -> BExp  
  | band : BExp -> BExp -> BExp  
  | bnot : BExp -> BExp.
```

► Semantics:

```
Fixpoint beval (b : BExp) (env : Env) : bool :=  
  match b with  
  | btrue => true  
  | bfalse => false  
  | blessthan a1 a2 => Nat.leb (aeval a1 env) (aeval a2 env)  
  | band b1 b2 => andb (beval b1 env) (beval b2 env)  
  | bnot b' => negb (beval b' env)  
  end.
```

Boolean expressions

► Syntax:

```
Inductive BExp :=  
  | btrue  : BExp  
  | bfalse : BExp  
  | blessthan : AExp -> AExp -> BExp  
  | band : BExp -> BExp -> BExp  
  | bnot : BExp -> BExp.
```

► Semantics:

```
Fixpoint beval (b : BExp) (env : Env) : bool :=  
  match b with  
  | btrue => true  
  | bfalse => false  
  | blessthan a1 a2 => Nat.leb (aeval a1 env) (aeval a2 env)  
  | band b1 b2 => andb (beval b1 env) (beval b2 env)  
  | bnot b' => negb (beval b' env)  
  end.
```

Assignments

► Syntax:

```
Inductive Stmt :=  
| assignment : Var -> AExp -> Stmt.  
Notation "A ::= B" := (assignment A B) (at level 54).  
Check n ::= 100 .  
n ::= 100  
      : Stmt
```

► Assignments modify the environment

► Semantics:

```
Fixpoint eval (s : Stmt) (env : Env) : Env :=  
match s with  
| assignment x a => update x (aeval a env) env  
end.
```

► (eval (n ::= 100) env) is of type Env!

Assignments

► Syntax:

```
Inductive Stmt :=  
| assignment : Var -> AExp -> Stmt.  
Notation "A ::= B" := (assignment A B) (at level 54).  
Check n ::= 100 .  
n ::= 100  
      : Stmt
```

► Assignments modify the environment

► Semantics:

```
Fixpoint eval (s : Stmt) (env : Env) : Env :=  
match s with  
| assignment x a => update x (aeval a env) env  
end.
```

► (eval (n ::= 100) env) is of type Env!

Assignments

► Syntax:

```
Inductive Stmt :=  
| assignment : Var -> AExp -> Stmt.  
Notation "A ::= B" := (assignment A B) (at level 54).  
Check n ::= 100 .  
n ::= 100  
      : Stmt
```

► Assignments modify the environment

► Semantics:

```
Fixpoint eval (s : Stmt) (env : Env) : Env :=  
match s with  
| assignment x a => update x (aeval a env) env  
end.
```

► (eval (n ::= 100) env) is of type Env!

Assignments

► Syntax:

```
Inductive Stmt :=  
| assignment : Var -> AExp -> Stmt.  
Notation "A ::= B" := (assignment A B) (at level 54).  
Check n ::= 100 .  
n ::= 100  
      : Stmt
```

► Assignments modify the environment

► Semantics:

```
Fixpoint eval (s : Stmt) (env : Env) : Env :=  
match s with  
| assignment x a => update x (aeval a env) env  
end.
```

► (eval (n ::= 100) env) is of type Env!

Assignments

► Syntax:

```
Inductive Stmt :=  
| assignment : Var -> AExp -> Stmt.  
Notation "A ::= B" := (assignment A B) (at level 54).  
Check n ::= 100 .  
n ::= 100  
      : Stmt
```

► Assignments modify the environment

► Semantics:

```
Fixpoint eval (s : Stmt) (env : Env) : Env :=  
match s with  
| assignment x a => update x (aeval a env) env  
end.
```

► (eval (n ::= 100) env) is of type Env!

Programs: sequence of statements

► Syntax:

```
Inductive Stmt :=
```

```
...
```

```
| seq s1 s2 => eval s2 (eval s1 env)
```

```
...
```

```
Notation "S S'" := (assignment S S') (at level 54).
```

```
Check n ::= 100 ;; i ::= 7 .
```

```
n ::= 100 ;; i ::= 7
```

```
      : Stmt
```

► Semantics:

```
Fixpoint eval (s : Stmt) (env : Env) : Env :=
```

```
match s with
```

```
...
```

```
| seq s1 s2 => eval s2 (eval s1 env)
```

```
end.
```


Programs: sequence of statements

► Syntax:

```
Inductive Stmt :=
```

```
...
```

```
| seq s1 s2 => eval s2 (eval s1 env)
```

```
...
```

```
Notation "S S'" := (assignment S S') (at level 54).
```

```
Check n ::= 100 ;; i ::= 7 .
```

```
n ::= 100 ;; i ::= 7
```

```
      : Stmt
```

► Semantics:

```
Fixpoint eval (s : Stmt) (env : Env) : Env :=
```

```
match s with
```

```
...
```

```
| seq s1 s2 => eval s2 (eval s1 env)
```

```
end.
```

Programs: sequence of statements

► Syntax:

```
Inductive Stmt :=  
  ...  
  | seq s1 s2 => eval s2 (eval s1 env)  
  ...  
Notation "S S'" := (assignment S S') (at level 54).  
Check n ::= 100 ;; i ::= 7 .  
n ::= 100 ;; i ::= 7  
      : Stmt
```

► Semantics:

```
Fixpoint eval (s : Stmt) (env : Env) : Env :=  
  match s with  
  ...  
  | seq s1 s2 => eval s2 (eval s1 env)  
  end.
```

Loops

► Syntax:

```
Inductive Stmt :=  
...  
| while : BExp -> Stmt -> Stmt.  
...
```

► Semantics:

```
Fixpoint eval (s : Stmt) (env : Env) : Env :=  
match s with  
...  
| while b s' => if (beval b env)  
    then (eval (seq s' (while b s')) env)  
    else env  
end.
```

► ERROR:

Error: Cannot guess decreasing argument of fix.

► Solution: define eval as a *relation*!

Loops

► Syntax:

```
Inductive Stmt :=  
...  
| while : BExp -> Stmt -> Stmt.  
...
```

► Semantics:

```
Fixpoint eval (s : Stmt) (env : Env) : Env :=  
match s with  
...  
| while b s' => if (beval b env)  
    then (eval (seq s' (while b s')) env)  
    else env  
end.
```

► ERROR:

Error: Cannot guess decreasing argument of fix.

► Solution: define `eval` as a *relation*!

Loops

► Syntax:

```
Inductive Stmt :=  
...  
| while : BExp -> Stmt -> Stmt.  
...
```

► Semantics:

```
Fixpoint eval (s : Stmt) (env : Env) : Env :=  
match s with  
...  
| while b s' => if (beval b env)  
    then (eval (seq s' (while b s')) env)  
    else env  
end.
```

► ERROR:

Error: Cannot guess decreasing argument of fix.

► Solution: define `eval` as a *relation*!

Evaluation as a relation

► **Assignment:**
$$\frac{\text{aeval } a \ E = v}{(\text{eval } (x ::= a) \ E \ (x \mapsto v ; E))}$$

► **Sequence:**
$$\frac{\text{eval } s_1 \ E_1 \ E' \quad \text{eval } s_2 \ E' \ E_2}{(\text{eval } (\text{seq } s_1 \ s_2) \ E_1 \ E_2)}$$

► **Loop (true case):**

$$\frac{\text{beval } b \ E_1 = \text{true} \quad \text{eval } s \ E_1 \ E' \quad \text{eval } (\text{while } b \ s) \ E' \ E_2}{(\text{eval } (\text{while } b \ s) \ E_1 \ E_2)}$$

► **Loop (false case):**
$$\frac{\text{beval } b \ E = \text{false}}{(\text{eval } (\text{while } b \ s) \ E \ E)}$$

Evaluation as a relation

► **Assignment:**
$$\frac{\text{aeval } a \ E = v}{(\text{eval } (x ::= a) \ E \ (x \mapsto v ; E))}$$

► **Sequence:**
$$\frac{\text{eval } s_1 \ E_1 \ E' \quad \text{eval } s_2 \ E' \ E_2}{(\text{eval } (\text{seq } s_1 \ s_2) \ E_1 \ E_2)}$$

► **Loop (true case):**

$$\frac{\text{beval } b \ E_1 = \text{true} \quad \text{eval } s \ E_1 \ E' \quad \text{eval } (\text{while } b \ s) \ E' \ E_2}{(\text{eval } (\text{while } b \ s) \ E_1 \ E_2)}$$

► **Loop (false case):**
$$\frac{\text{beval } b \ E = \text{false}}{(\text{eval } (\text{while } b \ s) \ E \ E)}$$

Evaluation as a relation

► **Assignment:**
$$\frac{\text{aeval } a \ E = v}{(\text{eval } (x ::= a) \ E \ (x \mapsto v ; E))}$$

► **Sequence:**
$$\frac{\text{eval } s_1 \ E_1 \ E' \quad \text{eval } s_2 \ E' \ E_2}{(\text{eval } (\text{seq } s_1 \ s_2) \ E_1 \ E_2)}$$

► **Loop (true case):**

$$\frac{\text{beval } b \ E_1 = \text{true} \quad \text{eval } s \ E_1 \ E' \quad \text{eval } (\text{while } b \ s) \ E' \ E_2}{(\text{eval } (\text{while } b \ s) \ E_1 \ E_2)}$$

► **Loop (false case):**
$$\frac{\text{beval } b \ E = \text{false}}{(\text{eval } (\text{while } b \ s) \ E \ E)}$$

Evaluation as a relation

► Assignment:
$$\frac{\text{aeval } a \ E = v}{(\text{eval } (x ::= a) \ E \ (x \mapsto v ; E))}$$

► Sequence:
$$\frac{\text{eval } s_1 \ E_1 \ E' \quad \text{eval } s_2 \ E' \ E_2}{(\text{eval } (\text{seq } s_1 \ s_2) \ E_1 \ E_2)}$$

► Loop (true case):

$$\frac{\text{beval } b \ E_1 = \text{true} \quad \text{eval } s \ E_1 \ E' \quad \text{eval } (\text{while } b \ s) \ E' \ E_2}{(\text{eval } (\text{while } b \ s) \ E_1 \ E_2)}$$

► Loop (false case):
$$\frac{\text{beval } b \ E = \text{false}}{(\text{eval } (\text{while } b \ s) \ E \ E)}$$

Bibliography

- ▶ Chapter Simple Imperative Programs in **Software Foundations - Volume 1**, Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hrițcu, Vilhelm Sjöberg, Andrew Tolmach, Brent Yorgey
<https://softwarefoundations.cis.upenn.edu/lf-current/Imp.html>